Python Data Structures: Lists, Tuples, Dictionaries and Sets

Lists

A list is an ordered, mutable sequence. Common methods are append(x), extend(iterable), insert(i, x), remove(x), pop(i), index(x), count(x), sort() and reverse(). Methods that change the list in place such as append, sort and reverse return None.

```
a = [1, 2, 3]
a.append(4)         # [1, 2, 3, 4]
a.pop()             # removes and returns 4
b = a.copy()        # independent shallow copy
c = a               # NOT a copy, both names refer to one list
```

Slicing uses seq[start:stop:step] and never raises IndexError, even when the range is out of bounds. Plain indexing does raise IndexError.

List comprehensions

A list comprehension builds a list in one expression: [expression for item in iterable if condition]. It is shorter and usually faster than appending in a loop.

```
squares = [n * n for n in range(1, 6)]     # [1, 4, 9, 16, 25]
evens = [n for n in range(10) if n % 2 == 0]
```

Tuples

A tuple is an ordered, immutable sequence. Trying to change an item raises TypeError: 'tuple' object does not support item assignment. A single item tuple needs a trailing comma, as in (5,).

Dictionaries

A dictionary maps unique, hashable keys to values. Use d[key] to read (KeyError if missing), d.get(key, default) for a safe read, d.items() to loop over pairs, d.keys() and d.values() for views, and d.setdefault or collections.defaultdict to initialise missing keys. Lists cannot be dictionary keys because they are unhashable.

```
scores = {"asha": 90, "ravi": 75}
scores["meera"] = 88
for name, mark in scores.items():
    print(name, mark)

if "kiran" in scores:
    print(scores["kiran"])

# Modifying a dict while iterating raises RuntimeError:
# dictionary changed size during iteration. Loop over list(d) instead.
```

Sets

A set is an unordered collection of unique hashable items. It supports union (|), intersection (&), difference (-) and fast membership testing with in. Converting a list to a set removes duplicates but loses order.

Mutability and copying

Assigning one list to another name does not copy it. Both names point to the same object, so changes through one are visible through the other. Use list.copy(), slicing [:] or list(x) for a shallow copy, and copy.deepcopy for nested structures. A common bug is creating a grid with [[0] * 3] * 3, which repeats the same inner list three times.

```
grid = [[0] * 3] * 3
grid[0][0] = 1
print(grid)              # [[1,0,0],[1,0,0],[1,0,0]]  surprise

grid = [[0] * 3 for _ in range(3)]   # fix
```